

CODE LAYOUT, READABILITY AND REUSABILITY

Project Name: ComicCraft – AI Powered Comic Story Generator

Team ID: _____

Code Quality Checklist

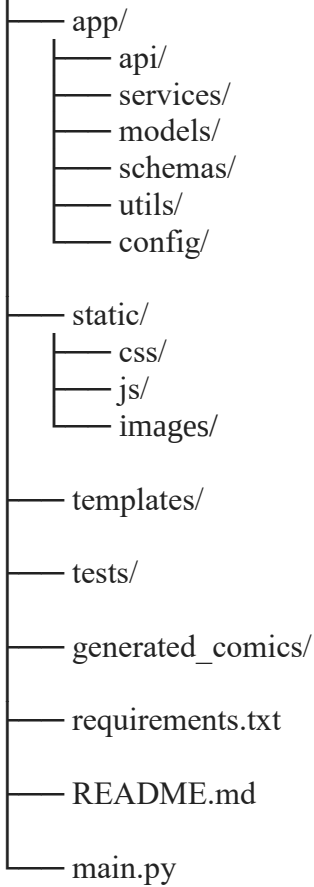
S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing and indentation are maintained throughout the codebase.	Yes	Improves readability and code consistency.
2	Proper File Structure	Files and folders are logically organized according to application layers.	Yes	Frontend, Backend, AI Services, Database, and Utilities are separated.
3	Meaningful Variable Names	Variables clearly represent their purpose and usage.	Yes	Enhances code understanding and maintenance.
4	Function / Method Names	Functions are named descriptively based on their responsibilities.	Yes	Improves code readability and debugging.
5	Code Comments	Inline and block comments explain important logic and workflows.	Yes	Useful for future maintenance and collaboration.
6	Modular Design	Code is divided into reusable modules and services.	Yes	Supports scalability and reusability.
7	No Redundant Code	Duplicate and unused code has been removed.	Yes	Reduces complexity and improves maintainability.
8	Error Handling	Exceptions and unexpected errors are handled properly.	Yes	Ensures application stability and reliability.

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	Authentication Module	Python, FastAPI	User Registration, Login, Session Management	High
2	Story Generation Service	Python, Gemini API	Comic Story Generation, Dialogue Creation, Narration Generation	High
3	Prompt Processing Module	Python	Input Validation, Prompt Enhancement	High
4	Image Generation Service	Python, Hugging Face API	Comic Panel Illustration Generation	High
5	PDF Generation Module	Python, FPDF	Comic Export, Report Generation	High
6	Comic Layout Engine	Python, Pillow	Comic Rendering and Formatting	High
7	Database Access Layer	PostgreSQL, SQLAlchemy	User Data, Comic Metadata, History Management	High
8	Frontend UI Components	HTML, CSS, JavaScript	Forms, Comic Preview, Download Interface	Medium
9	Error Handling Utilities	Python	API Responses, Validation Errors, Exception Logging	High
10	Configuration Module	Python	API Keys, Environment Variables, Application Settings	High

Folder Structure Followed

ComicCraft/



Readability Practices Followed

Naming Conventions

- Meaningful variable names
- Descriptive function names
- Consistent class naming

Examples:

```
generate_comic_story()
generate_panel_images()
create_pdf_comic()
save_comic_metadata()
```

Commenting Standards

- Function-level documentation
- API endpoint descriptions
- Inline comments for complex logic

Formatting Standards

- PEP 8 coding guidelines
 - Consistent indentation
 - Proper spacing and line breaks
-

Reusability Practices Followed

Service-Oriented Design

Each major feature is implemented as an independent service:

- Story Generation Service
- Image Generation Service
- PDF Service
- Authentication Service

Utility Functions

Common functionality is placed inside reusable utility modules.

Configuration Management

API keys and environment settings are stored separately from business logic.

Database Abstraction

Database operations are centralized to allow easy migration and maintenance.

Error Handling Strategy

The application handles:

- Invalid user inputs
- API failures
- Image generation errors
- Database connection failures
- PDF generation issues

Error messages are logged and displayed in a user-friendly manner.

Overall Code Quality Assessment

Aspect	Rating (1–5)	Comments
Code Layout & Structure	5	Well-organized layered architecture with clear separation of concerns.
Readability	5	Consistent naming conventions, formatting, and documentation.
Reusability	5	Modular services and reusable components across the application.
Documentation / Comments	4	Adequate documentation and comments for maintainability.
Overall Score	4.75 / 5	High-quality, maintainable, scalable, and reusable codebase.

Conclusion

The ComicCraft codebase follows best practices for layout, readability, and reusability. The project adopts a modular architecture, meaningful naming conventions, reusable service components, and robust error handling mechanisms. These practices ensure maintainability, scalability, and ease of future enhancements while supporting efficient collaboration among development team members.